Lecture 16: Intro to ML, Feature Engineering

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

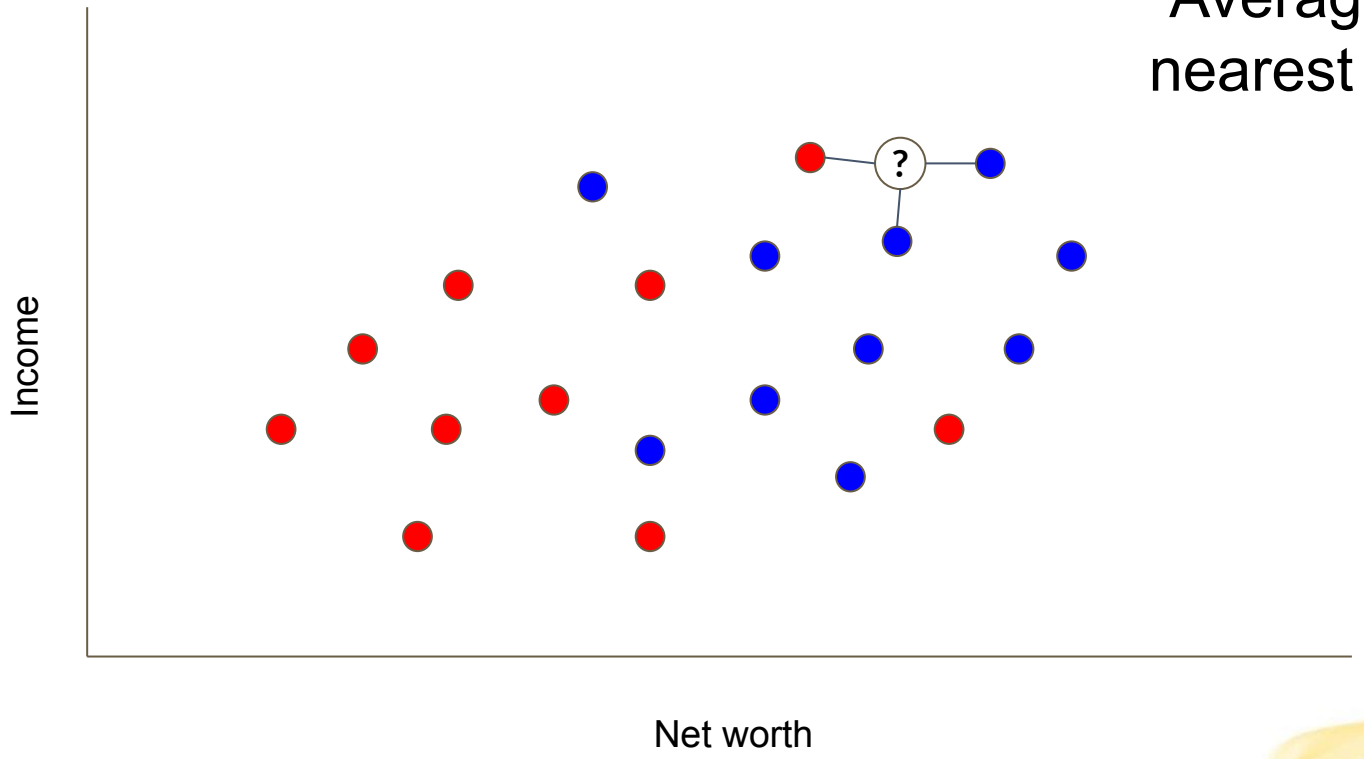
- PA4, WA4 due this Sunday, March 29

KNN

Motivating Example: Bank Loan Algorithm

Plot the datapoint:

- Loan not paid
- Loan paid



<u>Take an average:</u>
"Average" the 3 nearest neighbors.

Our Example Bank Loan Algorithm: KNN

- Our previous Bank Loan example was actually a simple implementation of the K-Nearest Neighbors (KNN) **classification** algorithm
- KNN is simple, easy-to-implement **supervised** machine learning algorithm.
  - Predicts outcomes based on similarity or proximity between data points.
- Different variations of KNN can be used to solve both **classification** and **regression** problems.
  - Regression → Predict continuous numerical values
  - Classification → Predict a category

Intuition for Nearest Neighbor Classification

Simple idea

- Store all training examples
- Classify new examples based on most similar training examples

Remember, KNN predicts outcomes based on the **similarity or proximity between data points**.

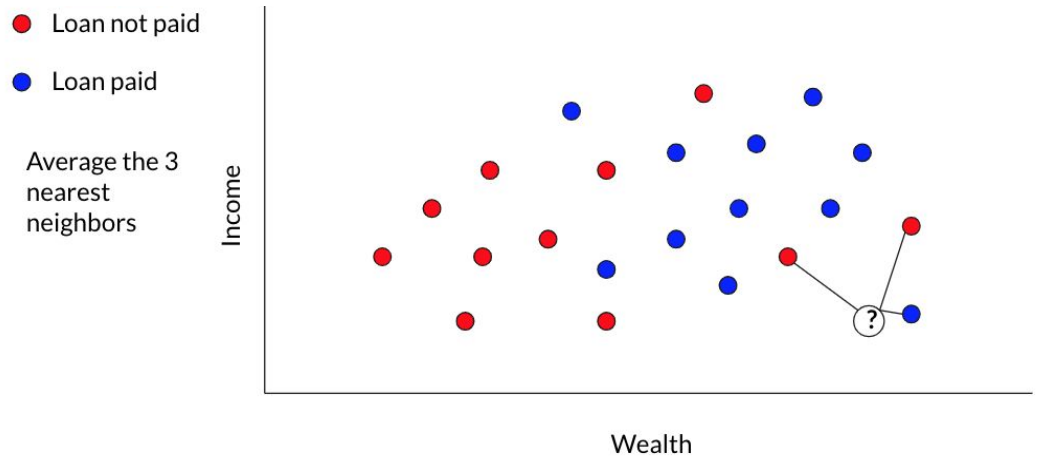
KNN Bank Loan Classification

Given the set K of the K nearest neighbors and point p:

- KNN Algorithm: identify the K most similar individuals (neighbors) to the person for whom we want to make a prediction (point p).

Let's make labels numeric:

- Label -1: Represents someone who did not pay back the loan.
- Label 1: Represents someone who did pay back the loan.



Variations on KNN: Weighted voting

Weighted Voting Default

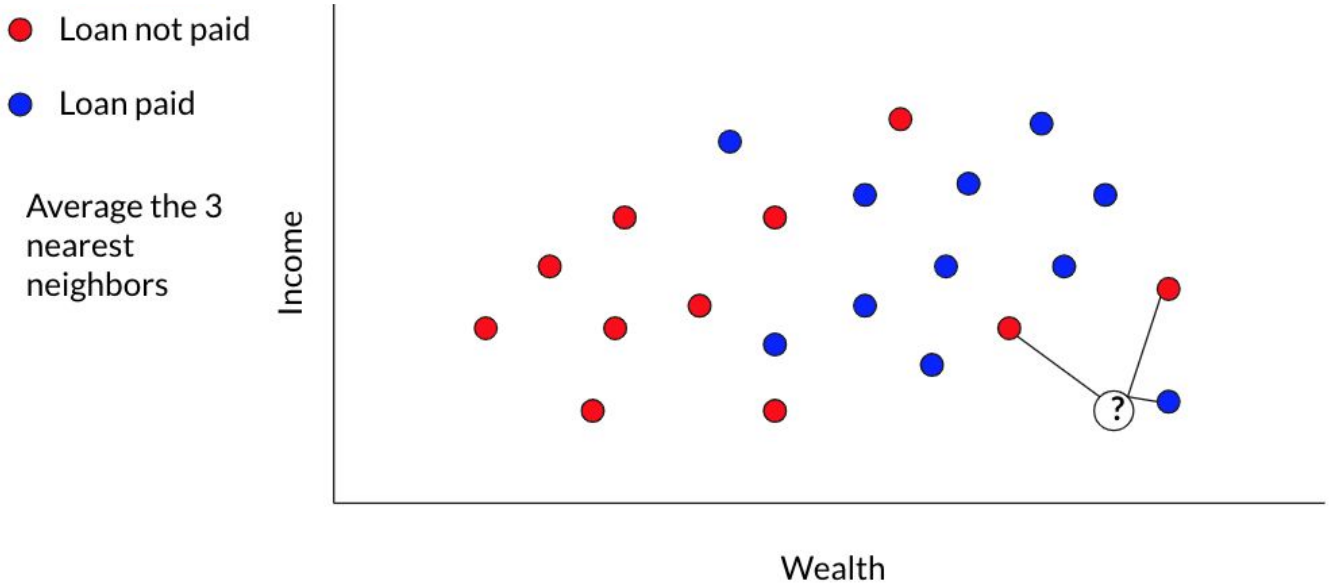
- All neighbors have equal weight
- Extension: weight neighbors by (inverse) distance
  - NEXT TOPIC: Weighted KNN

A Modified Version: Weighted KNN

In **Standard KNN**, prediction is based on the majority class among the K nearest data points and **equal weight** is given to **all K nearest neighbors**.

- Ex: If K=5 and the nearest neighbors are {1,1,1,−1,−1}, the prediction would be 1 (majority class).

Weighted KNN is similar to KNN, but the nearest k points are assigned a weight based on their distance.

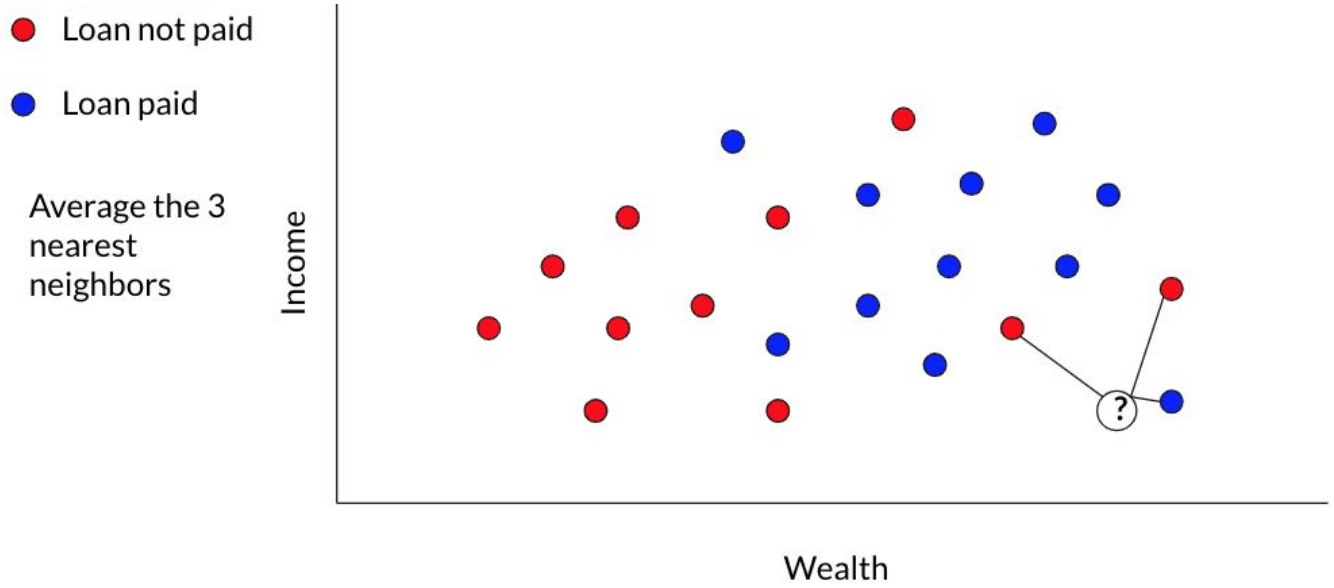


A Modified Version: Weighted KNN

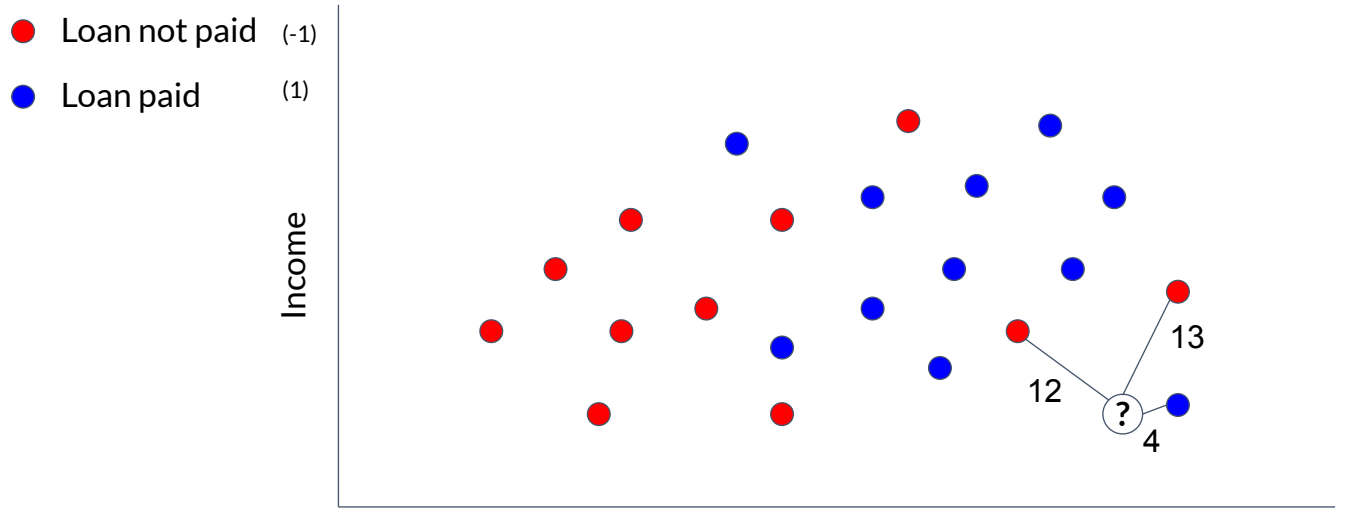
In **Standard KNN**, prediction is based on the majority class among the K nearest data points and **equal weight** is given to **all K nearest neighbors**.

- Ex: If K=5 and the nearest neighbors are $\{1,1,1,-1,-1\}$, the prediction would be 1 (majority class).

The intuition behind weighted KNN is to give more weight to the points which are nearby and less weight to the points which are farther away.



KNN Bank Loan Classification



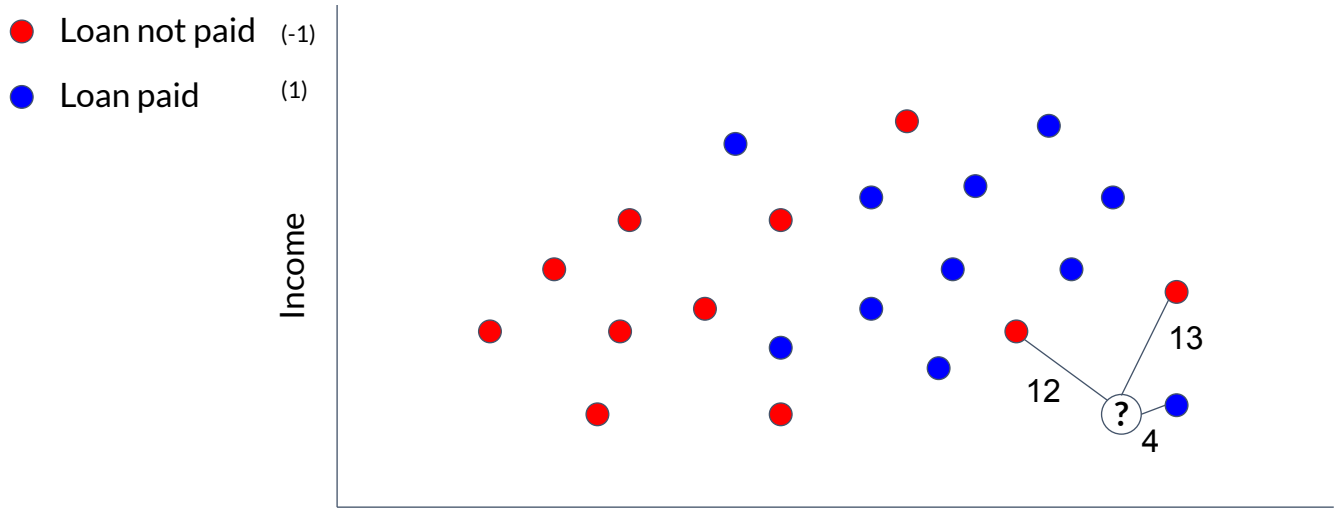
"Votes" for red:

$$\frac{1}{12} + \frac{1}{13} = \frac{25}{156} \approx 0.16$$

"Votes" for blue:

$$\frac{1}{4} = 0.25$$

KNN Bank Loan Classification



Input

$$\frac{1}{12}\times(-1)+\frac{1}{13}\times(-1)+\frac{1}{4}\times 1$$

Exact result

$$\frac{7}{78}$$

Decimal approximation

0.089743589743589743589743589

$$y = (\frac{1}{bigDistance} * -1) + (\frac{1}{bigDistance} * -1) + (\frac{1}{smallDistance} * 1)$$

Summary: KNN vs Weighted KNN

KNN	Weighted KNN
It treats **all neighbors equally**	Instead of treating all k-nearest neighbors equally, **assign different weights** to them based on their distance from the query point.
Classification tasks: count the occurrences of each class among the k-nearest neighbors and choose the class with the **highest count (mode)** as the predicted class.	**Classification tasks**: when determining the predicted class, the contributions of neighbors are scaled by their weights. *Closer neighbors have a stronger influence on the prediction.*
Regression tasks: take the **average (mean)** of the values associated with the k-nearest neighbors as the predicted value.	**Regression tasks**: the values associated with neighbors are multiplied by their weights before averaging, giving *more importance to closer neighbors in the prediction*.

Decision Boundary of a Classifier

The line that separates positive and negative regions in the feature space.

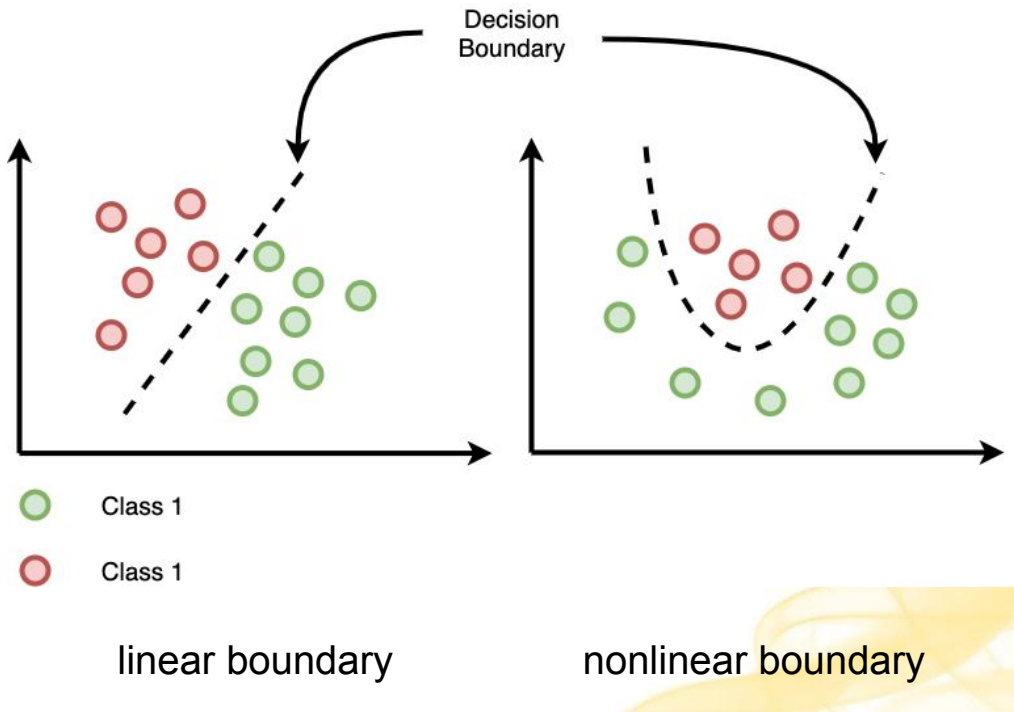
Purpose: It determines how new data points are classified based on their features.

Types:

- **Linear Boundary:** Straight line that separates classes in a linearly separable problem.
- **Non-linear Boundary:** Complex shapes that separate classes in non-linearly separable problems.

Why is it useful?

- Helps us visualize how examples will be classified for the entire feature space
- Helps us visualize the complexity of the learned model



KNN Boundary Conditions

What if K=1?

- Our KNN model considers only at 1 nearest neighbor (the closest) when making predictions.
  - **Decision Boundary:** The decision boundary can be irregular and jagged because the model is sensitive to individual data points.
- Our model will basically be almost memorizing things - it will just say whatever is closest.
  - **Doesn't consider any broader patterns or relationships in the data.**

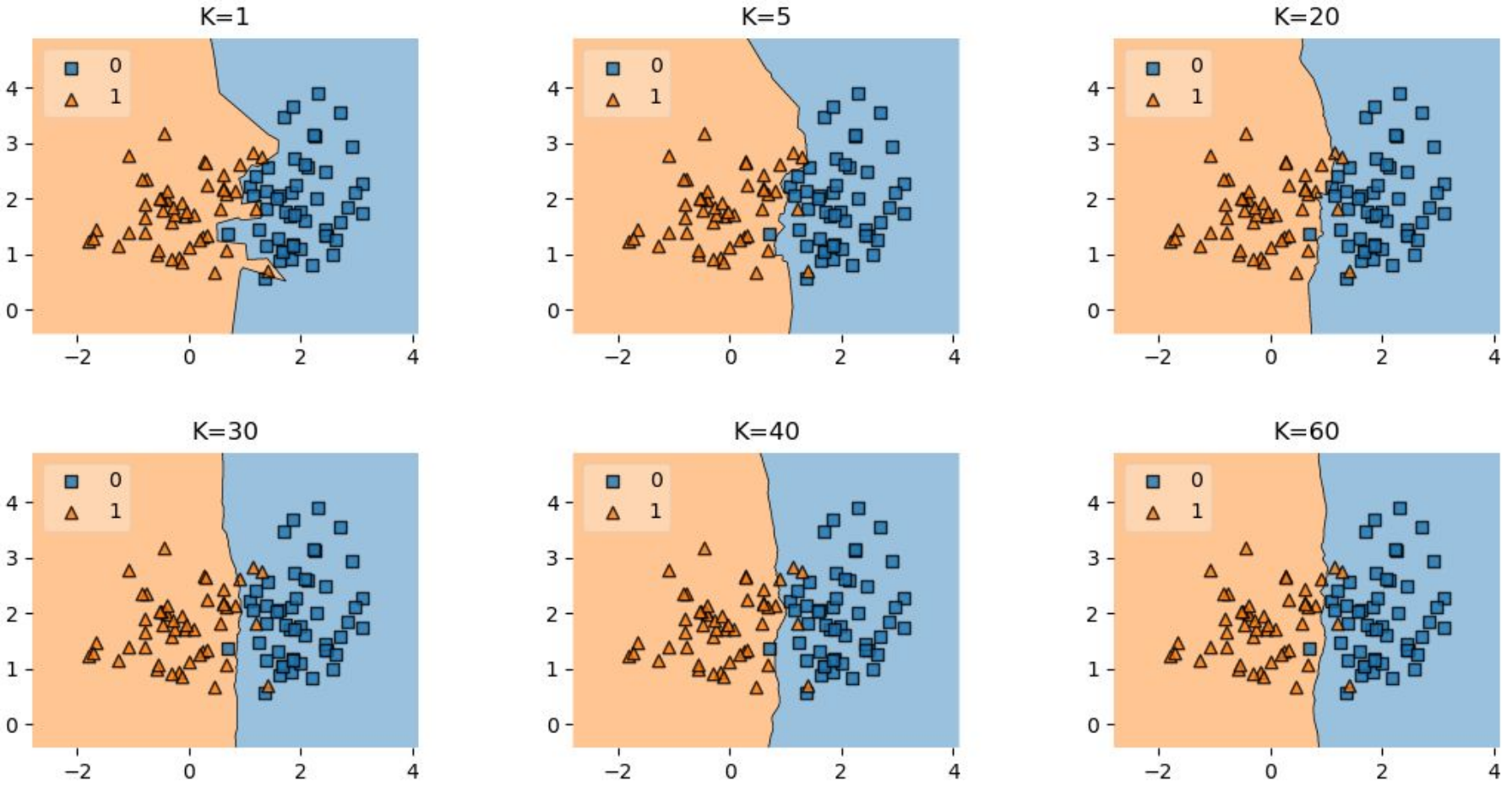
What if K=N?

KNN Boundary Conditions

What if K=N?

- Our KNN model **considers all data points as neighbors**, effectively taking into account the entire dataset.
  - It treats all data points as relevant neighbors, regardless of their actual similarity to the query point.
  - It ignores the local structure of the data.
- Soon, more and more irrelevant points will contribute, and we will just predict the mode of the data for any new data point.

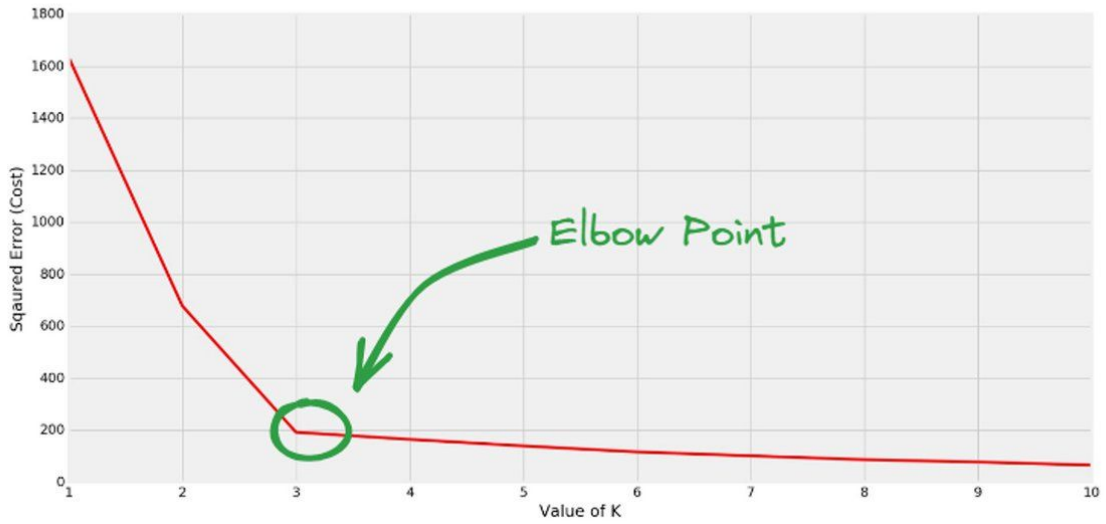
KNN Boundary Conditions Visualized



How to Choose K in KNN

One way: Try multiple Ks and graph the variation, then pick the lowest inflection point, or elbow point.

- This is known as the **Elbow Method**
- The inflection point is not crisply defined, but this can be a useful way to identify an appropriate K value for your specific problem.



Pros and Cons of K-Nearest Neighbor

Pros

- **Pretty accurate**: For many applications, KNN does a great job. Used successfully in large-scale production in industry for a variety of real-life problems.
- **Easy to implement**: Given the algorithm's simplicity and accuracy, it is one of the easiest to implement and one of the first classifiers a new data scientist will learn.
- **Easy to understand**: Very simple rule set that is highly explainable. All you need to understand is distance and how you got k.
- **Few hyperparameters**: KNN only requires a k value and a distance metric, which is low compared to other machine learning algorithms.
- **Lazy learning**: KNN is a **lazy algorithm**, meaning that it doesn't have a training step because it stores the entire dataset in memory and uses it directly to make predictions.

Pros and Cons of K-Nearest Neighbor

Cons

- **Resources**: Requires higher memory and has higher computational cost compared to other classifiers.
  - Needs to store and process the entire training set, which can be very large (on the order of petabytes or exabytes).
- **Tuning**: K must be chosen/tuned correctly and a proper distance metric chosen.
  - Choice of K impacts performance of the KNN model and needs to be selected carefully
    - K too small → noise and outliers effect
    - K too large → lose relevant info for effective choice of boundary
  - Choice of distance metric must be appropriate for the specific problem

Summary: Vocabulary for KNN

- KNN is a **supervised learning** model, which means the **training data is labeled**.
  - For our example, we had labels for whether or not a person had paid back their loans.
- Our bank loan example was a **classification** problem, where we were deciding between categories
  - We sorted things into categories: we were deciding if someone belongs to one of two groups (Pay Back or Not).
- We did **binary classification** where there are only two categories.
- KNN is an **instance-based model**, meaning it requires all the training data be stored in memory to work.
- We chose appropriate **hyperparameters**, which are the parameters that need to be **tuned** ahead of time in order to run the algorithm
  - For KNN, is simply the value of K.
  - We also chose a distance metric, though that didn't need to be tuned

Python Code Sample for KNN

1. Import necessary modules

```
from sklearn.neighbors import KNeighborsClassifier
```

2. Load your labeled training data
3. Create a KNN model (for a predetermined value of K; default distance is Euclidean) and fit it to the training data:

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(data, labels)
```

4. Make a prediction on new data points that the model has not seen before

```
prediction = knn.predict(new_point)
```

Other algorithms

There are many, many ML algorithms.

And many, many varieties of each algorithm.

Here are *some* of them:

Averaged one-dependence estimators
Artificial neural network
Convolutional neural network
Extreme learning machine
Feedforward neural network
Logic learning machine
Long short-term memory
Recurrent neural network
Self-organizing map
Bayesian networks
Boosting
Case-based reasoning
Conditional random field
C4.5 algorithm
C5.0 algorithm
Chi-squared automatic interaction detection
Classification and regression tree
Conditional decision tree
Decision stump
Decision tree
ID3 algorithm
Iterative dichotomiser 3
Random forest
SLIQ
Bootstrap aggregating
Boosting
Gaussian process regression
Gene expression programming
Group method of data handling
Inductive logic programming
Information fuzzy networks
Instance-based learning
Eclat algorithm

K-nearest neighbour
Lazy learning
Learning vector quantization
Elastic-net
Lasso
Linear discriminant analysis
Linear regression
Logistic regression
Multinomial logistic regression
Naive bayes classifier
Ordinary least squares
Passive aggressive algorithms
Perceptron
Polynomial regression
Ridge regression / classification
Support vector machine
Logistic model tree
Analogical modelling
Nearest neighbour algorithm
Ordinal classification
Probably approximately correct learning
Quadratic classifiers
Random forests
Ripple down rules
Symbolic machine learning
Active learning
Co-training
Graph-based methods
Generative models
Low-density separation
Transduction
Apriori algorithm

FP-growth algorithm
Auto-encoders
BIRCH
Conceptual clustering
DBSCAN
Expectation-maximization
Fuzzy clustering
Hierarchical clustering
K-means clustering
K-medians
Mean-shift
OPTICS algorithm
Single-linkage clustering
Canonical correlation analysis
Dynamic mode decomposition
Factor analysis
Feature extraction
Feature selection
Independent component analysis
Linear discriminant analysis
Multidimensional scaling
Non-negative matrix factorization
Partial least squares regression
Principal component analysis
Principal component regression
Projection pursuit
Sammon mapping
T-distributed stochastic neighbour embedding
Expectation-maximization algorithm
Generative topographic map
Information bottleneck method
Manifold learning

Deterministic policy gradient
Learning automata
Proximal policy optimization
Q-learning
Soft actor-critic
State–action–reward–state–action
Temporal difference learning
Trust region policy Optimization
Other

Bayesian belief network
Bayesian knowledge base
Deep belief networks
Deep boltzmann machines
Deep neural networks
Discrepancy modelling
Gaussian naive bayes
Generative adversarial network
Hierarchical temporal memory
Knowledge-enhanced machine learning
Markov models
Multinomial naive bayes
Neural style transfer
Physics-informed machine learning
Sparse identification of nonlinear dynamics
Transformer
Vector quantization

In this class

We'll look at a handful of the most common/important algorithms.

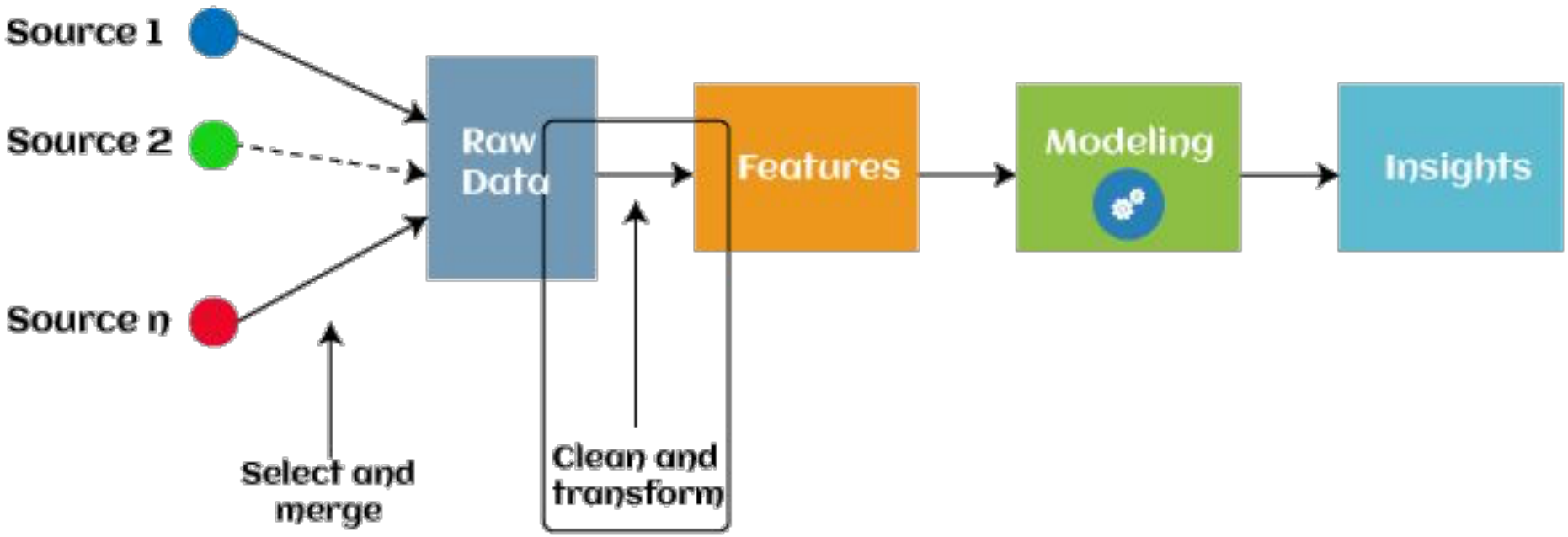
Understanding these will give you a good foundation and make it easier to learn others as needed.

No Free Lunch Theorem (NFL)

There is no single algorithm that will work well for all tasks.

Feature Engineering

Feature Engineering



Feature Engineering

The overall process of selecting, manipulating, and transforming raw or engineered data into a more effective set of input features for machine learning models.

It is a **pre-processing** step done **for machine learning**.

Areas of Feature Engineering:

Feature Creation	Feature Transformation	Feature Selection	Feature Extraction

***Goal**: enable the ML algorithm do the least work to produce the highest quality predictions possible*

Why do we need Feature Engineering?

- Improves model performance
- Lowers computational costs and time to train
  - In many cases, is the difference between being able to run the algorithm or not
- Improves model interpretability

Feature Engineering is critical because the **quality of features** directly impacts the **ML model's ability to learn patterns and make accurate predictions**.

Feature Creation

Feature Creation

The process of **generating new features** from existing ones, or from domain knowledge.

- Includes:
  - Combining features
  - Creating interaction terms
  - Extracting useful information from unstructured data (ex. large text, images).
- Done in order to provide the most useful features for the ML model.

<u>Examples</u>

- Date creation: # days since a reference day, day of week, holiday indicators
- Clustering (*future topic*): Creating features based on data similarity, using techniques like k-means

Feature Creation Example

House ID	Bedrooms	Bathrooms	Square Footage	Year Built	Sale Price
1	3	2	2000	1995	$300,000
2	4	2.5	2500	2005	$400,000
3	2	1.5	1500	1980	$250,000

Sample Data: House Prediction

Feature Creation Example

House ID	Bedrooms	Bathrooms	Square Footage	Year Built	Sale Price	Age of the House	Total Number of Rooms	Price per Square Foot
1	3	2	2000	1995	$300,000	28	5	$150/sq ft
2	4	2.5	2500	2005	$400,000	18	6.5	$160/sq ft
3	2	1.5	1500	1980	$250,000	43	3.5	$166.67/sq ft

These new features can help us understand more about our data.

Feature Transformation

Feature Transformation

The process of **modifying existing features** to make them more suitable for modeling.

Transforms can be **linear** (scaling or shifting values) or **nonlinear** (taking the logarithm or square root of a variable).

Common methods:

- Feature Scaling
- Log Transformation
- Encoding
- Discretization

Feature Scaling

Transforming the values of **independent variables** to have similar characteristics.

2 common techniques:

1. Normalization - similar range of values
2. Standardization - common distribution characteristics

Goal: Make sure features (mostly continuous features) are on almost the same scale (similar range) so that each feature is equally important and make it easier to process by most ML algorithms.

Feature Scaling Motivation: Euclidean Distance

Let, X=Salary, Y=Age.

Calculate Euclidean distance between rows 2 and 9.

Notice: $(x_2-x_1)^2 >> (y_2-y_1)^2$

X: 83k - 48k = 35k → $(x_2-x_1)^2$ = 1.2B

Y: 50 - 27 = 23 → $(y_2-y_1)^2$ = 529

→ the Euclidean distance will be **dominated by the salary** if we do not apply feature scaling.

	Country	Age	Salary	Purchased
1	France	44	72000	No
2	Spain	27	48000	Yes
3	Germany	30	54000	No
4	Spain	38	61000	No
5	Germany	40		Yes
6	France	35	58000	Yes
7	Spain		52000	No
8	France	48	79000	Yes
9	Germany	50	83000	No
10	France	37	67000	Yes

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Features with larger scales will contribute more to the distance calculation, potentially biasing the results towards those features

Normalization (Min-Max)

Changing the range of features by mapping them onto a fixed scale, using **minimum** and **maximum** values.

Common examples

- Mapping values onto the range [0, 1]
- Mapping values onto the range [0, 100]
- Mapping values onto the range [-1, 1]

[0, 1]

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)}$$

Why do it?

- To ensure features are on comparable scales
  - Some ML algorithms take into account how far apart data points are (such as KNN that we saw earlier). Without scaling they tend to be biased towards numerically larger values
- Can help certain ML algorithms converge faster
  - ex. gradient descent (*future topic*)
- Good as long as outliers aren't an issue

⚠️ sensitive to outliers

Normalization (Min-Max) – other scales

[0,1] is the standard normalization range.

However, once you have data on the [0,1] range, converting to other ranges is trivial. For example:

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)}$$

[0, 1] Normalized value

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)} \cdot 100$$

[0, 100] Normalized value

$$x_{norm} = \left(\frac{x_i - min(x)}{max(x) - min(x)} \right) \cdot 2 - 1$$

[-1, 1] Normalized value

Normalization (Min-Max) Example

Suppose we have a training dataset with:

- Income in dollars (ranges 0 to 12.53 billion)
- Age in years (ranges 0 to 100)

Through normalization, we have:

- Income: 0 → 0, 12.53 billion → 1
- Age: 0 → 0, 100 → 1

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)}$$

Here we are mapping all of our feature values onto the range of [0, 1]

Normalization (Min-Max) Example

Suppose we have a training dataset with:

- Income in dollars: 200,000, 300,000, 100,000, 400,000,
- Age in years: 40, 60, 30, 20

Through normalization, we have, in order

- Income:
- Age:

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)}$$

Normalization (Min-Max) Example

Suppose we have a training dataset with:

- Income in dollars: 200,000, 300,000, 100,000, 400,000
- Age in years: 40, 60, 30, 20

Through normalization, we have, in order

- Income: 0.33, 0.66, 0, 1
- Age: 0.5, 1, 0.25, 0

$$x_{norm} = \frac{x_i - min(x)}{max(x) - min(x)}$$

Normalization (Min-Max) Python Example

In Python, we can use scikit-learn library to apply **`MinMaxScaler`** (more efficient and faster than a custom implementation, especially for large datasets.)

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df[['feature-a']])

print("Original:", data)
print("Scaled:", scaled_data.ravel())
```

```
Original: [  2  45 -23  85  28   2  35 -12  99   0]
Scaled: [0.23 0.63 0.00 1.00 0.47 0.23 0.54 0.10]
```

Normalization (Min-Max) Common uses

Normalization is commonly used to assist ML algorithms, particularly those like K-Nearest Neighbors (KNN) and Neural Networks (*future topic*), which do not assume any specific distribution of the data (i.e. the distribution of your data is not necessarily Normal/Gaussian).

Normalization:

- Does not significantly alter the distribution of the feature.
- Ensures no single feature dominates the statistics.
  - Preserves relative distances between data points.

Standardization

Changing the scale of features using the **mean** and **standard deviation**.

- Subtract the distribution mean from each feature, then divide those values by the standard deviation.

Example

- Z-Score Normalization: Scaling features to have a mean of 0 and a standard deviation of 1.

$$z_i = \frac{x_i - \mu}{\sigma}$$

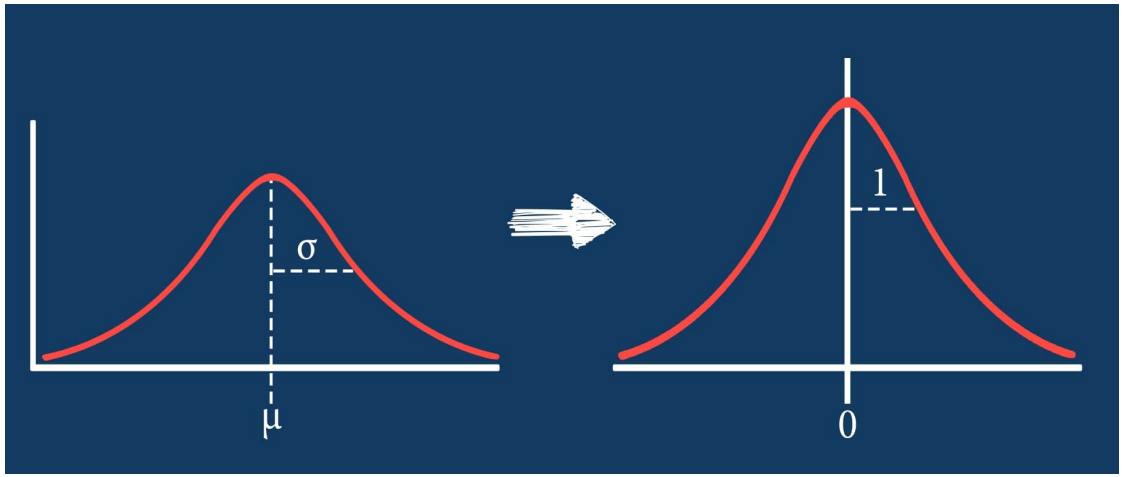
Why do it?

- To more easily compare distributions
  - Doesn't alter distribution shape and ensures that features have comparable scales.
- It's robust to outliers
- Many ML algorithms perform better with standard distributions

Z-Score Normalization (Standardization)

Features are rescaled to ensure the mean and the standard deviation to be 0 and 1, respectively (zero mean and unit variance)

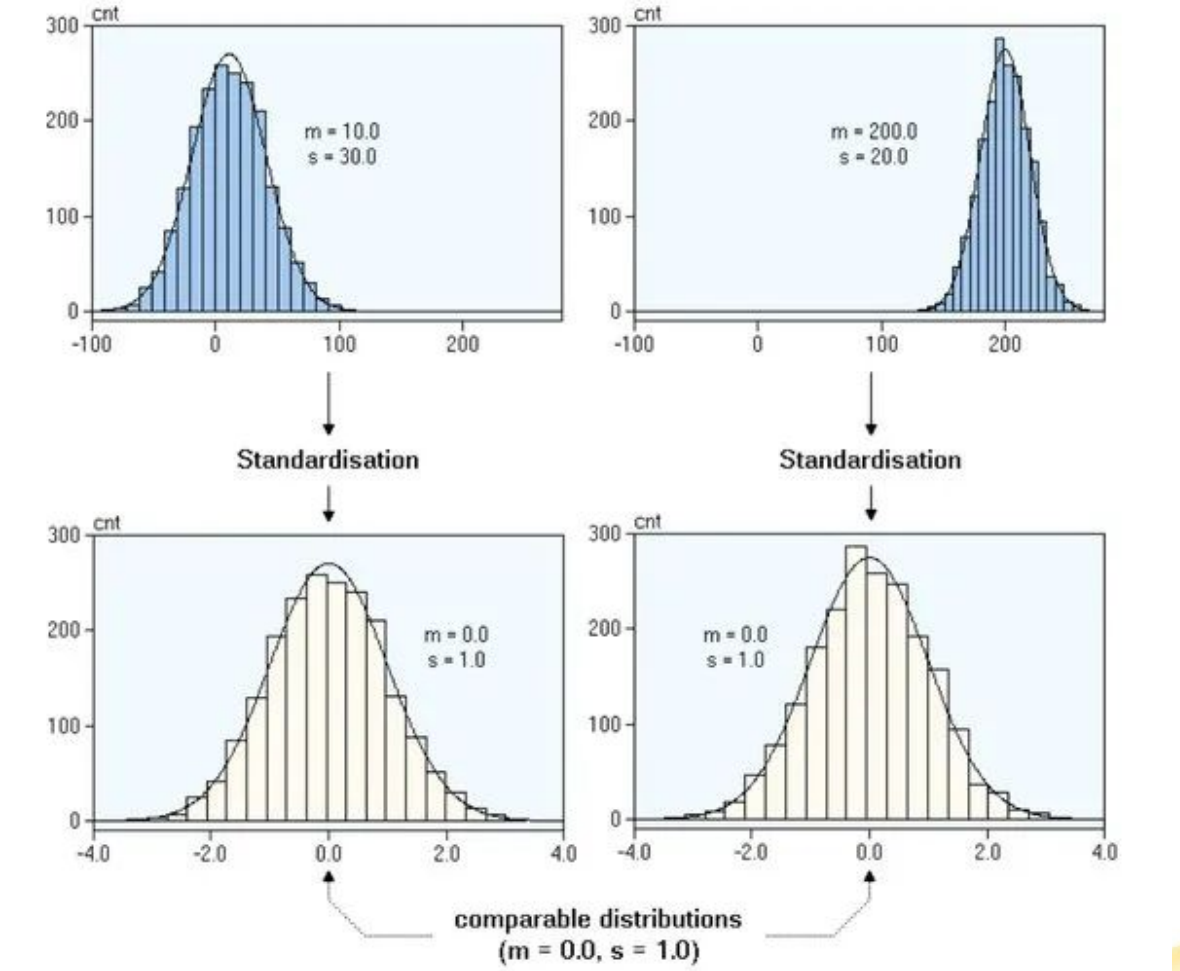
$$z_i = \frac{x_i - \mu}{\sigma}$$



Helpful for algorithms that **expect features to follow a standard normal distribution** or **use distance-based metrics**

- Results in better performance and convergence, where the algorithm reaches a stable or optimal solution.
- Algorithm Examples: KNN, PCA, SVM, etc

Z-Score Normalization (Standardization)



Z-Score Normalization Python Example

In Python, we can use scikit-learn library to apply **`StandardScaler`** (more efficient and faster than a custom implementation, especially for large datasets.)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[['feature-a']])

print("Original:", data)
print("Scaled:", scaled_data.ravel())
```

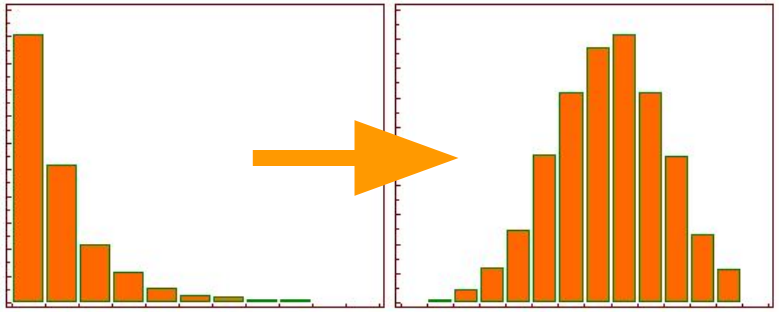
```
Original: [2.00 45.00 -23.00 85.00 28.00 2.00 35.00 -12.00]
Scaled: [-0.55 0.75 -1.31 1.97 0.24 -0.55 0.45 -0.98]
```

Standardization vs Normalization

Standardization is more robust to outliers, and in many cases, is **preferable** over Min-Max Normalization.

However, to get all the features into the same range, which is helpful for distance calculations (like with KNN), use Normalization

Log Transformation



Changing the skew of a feature by applying a **logarithmic function**.

Example

- Taking the log of income values

	Income	Age	Department	log_income
0	15000	25	HR	9.615805
1	1800	18	Legal	7.495542
2	120000	42	Marketing	11.695247
3	10000	51	Management	9.210340

Why do it?

- To convert a skewed distribution to a normal (or close to normal) distribution
- Helps reduce the impact of too low or too high values
- It can make the data more suitable for analysis techniques (t-test, Anova) that assume a more normal (bell-shaped) distribution of values.

Encoding

Convert categorical or text feature values into **numerical** values by applying an **encoding function**.

Examples

- One-Hot Encoding: Convert categorical variables into binary vectors (ex. 1 = feature is present, 0 = feature not present)
- Target-Mean Encoding: Replace categorical values with the mean of the target variable
- Frequency Encoding: Assign a numerical value to each feature based on how often it appears

Why do it?

- Many ML algorithms prefer or perform better with numerical values

Encoding Example: One Hot Encoding

Some Machine Learning methods hate categoricals!

- Jobs like "Nurse" or "Doctor" don't have an order.
- Labeling (like making Nurse 0 and Doctor 1) might make the model think one job is more important.
- One Hot Encoding makes each job a separate choice (binary feature) without a rank, yet enables us to represent categorical variables as numerical values.
- It treats all jobs fairly, without making one is better than another.
- It can be useful when data has no relationship to each other, or when the order of numbers is not significant.

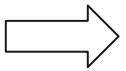
Index	Profession
1	Nurse
2	Doctor
3	Actor
4	Teacher
5	Nurse

Encoding Example: One Hot Encoding

One Hot Encoding is a data pre-processing technique that converts categorical variables into binary vectors:

- Each category becomes a binary column.
- "1" indicates the presence of a category; "0" indicates absence.

ID	Profession
1	Nurse
2	Doctor
3	Actor
4	Teacher
5	Nurse
6	Nurse



Index	Profession	Nurse	Doctor	Actor	Teacher
1	Nurse	1	0	0	0
2	Doctor	0	1	0	0
3	Actor	0	0	1	0
4	Teacher	0	0	0	1
5	Nurse	1	0	0	0
6	Nurse	1	0	0	0

More One Hot Encoding Examples

Color
Red
Green
Blue

One-hot encoding

d1	d2	d3
1	0	0
0	1	0
0	0	1

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

How to implement One-Hot Encoding

```python
import pandas as pd

# Sample data: A list of colors
data = {'ID': [1, 2, 3, 4],
        'Color': ['Red', 'Blue', 'Green', 'Red']}

df = pd.DataFrame(data)

# Apply One-Hot Encoding
encoded_df = pd.get_dummies(df, columns=['Color'])

print("Original Dataframe:")
print(df)
print("\nOne-Hot Encoded Dataframe:")
print(encoded_df)
```

```
Original Dataframe:
   ID  Color
0   1    Red
1   2   Blue
2   3  Green
3   4    Red

One-Hot Encoded Dataframe:
   ID  Color_Blue  Color_Green  Color_Red
0   1       False        False       True
1   2        True        False      False
2   3       False         True      False
3   4       False        False       True
```

One Hot Encoding

Upsides:

- You get to use your categorical variables
  - For models that require numerical input
- Solution when a categorical variable has no natural ordering
  - ex. “male”, “female”

Downsides:

- If there are 50 categories, your dataset explodes!
  - Dimensionality in the number of categories
  - Can lead to sparse data → most columns will probably have zeros
- Can lead to overfitting
  - Esp if there are many categories and small sample size

Discretization

Convert numerical features to discrete values.

Example

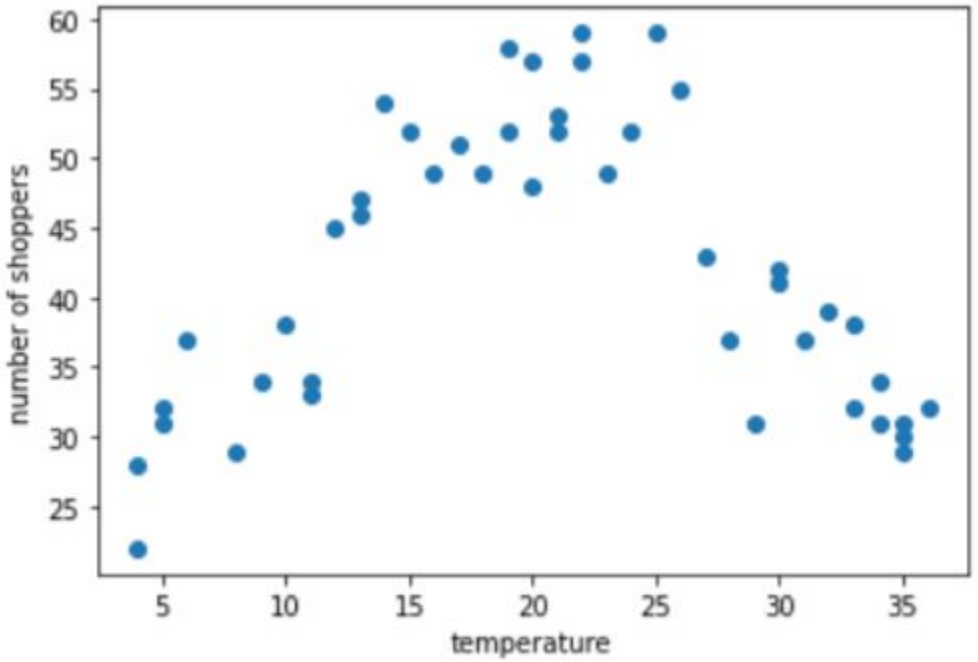
- Binning: Grouping continuous values into discrete bins or categories. (Usually turns numeric data into categorical data.)
  - Strategies:
    - Uniform: Bins have the same width of possible values
    - Clustered: Clusters are defined and observations are mapped
    - Quantile: Bins have the same number of values, and observations are mapped by quantiles

Why do it?

- Some ML algorithms prefer or require categorical or ordinal values
- Can simplify the problem, reduce errors, improve understandability, etc

Binning Example

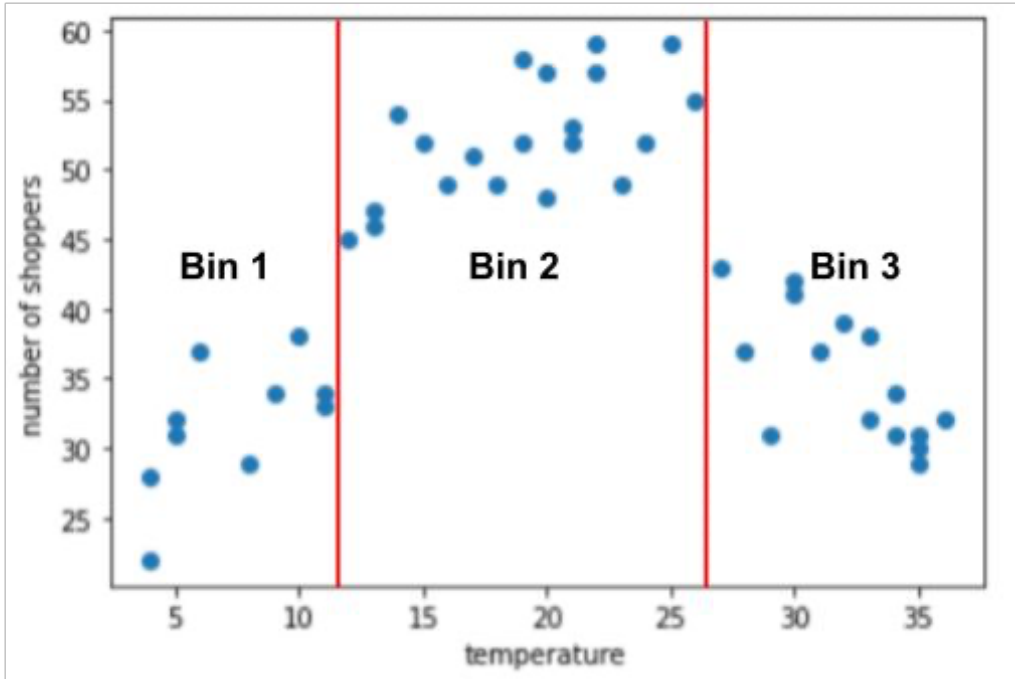
Number of shoppers versus temperature



Source: Google Developers

Binning Example

Number of shoppers versus temperature: bin temperature (say)



Source: Google Developers

Motivation
for
Feature Selection & Feature Extraction

Motivation: The Curse of Dimensionality

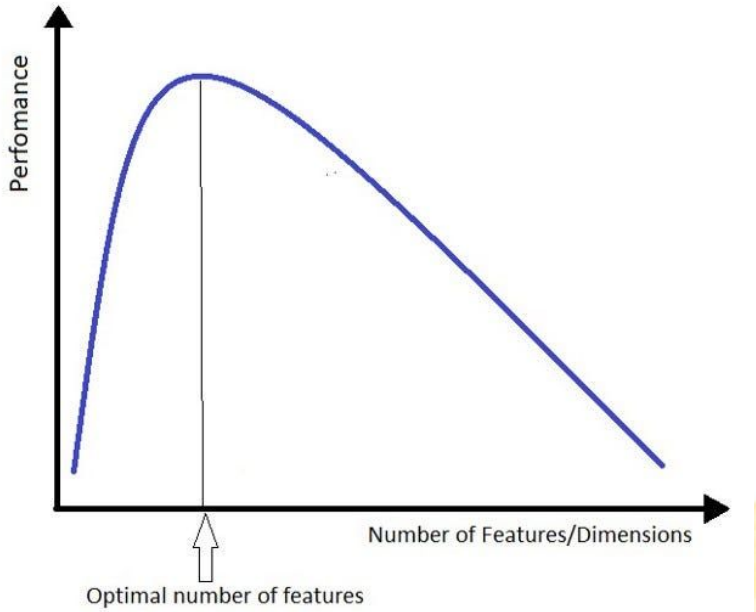
- Term first introduced by Richard Bellman in “Dynamic Programming” (1957)
- **Gist**: There are difficulties that arise in higher dimensional spaces that aren’t seen in lower dimensional spaces.
- **The challenge**: as dimensionality increases, so does the volume of the feature space.
  - This causes the available data to become sparse
    - The number of observations needed for the same level of accuracy increases!
    - But sample size usually doesn’t change.
  - More dimensions (features) tend to increase noise and errors
  - Complexity/run time increases with dimension
    - Many algorithms are $O(n \cdot d^2)$ or more!

The Curse of Dimensionality

Common Pitfall:

Assuming that adding more features will improve ability to solve the problem

In reality, using more features often leads to worse performance after some optimal number.



The Curse of Dimensionality

What can we do about it?

Dimensionality Reduction

The process of taking input data from high-dimensional space to low-dimensional space in a way that maintains key properties of the original dataset.

Methods:

1. Feature Selection
2. Feature Extraction

Feature Selection

The Good News

Most real-world applications have low-dimensional structure.

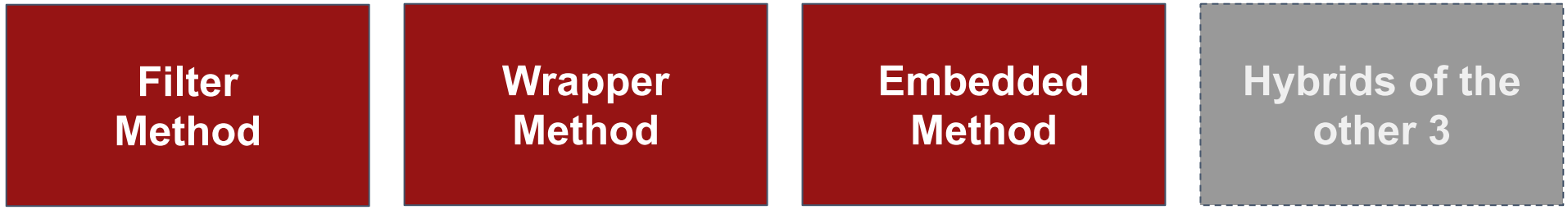
This means we can attempt to find the fewest features that matter the most for solving our problem.

Feature Selection

The process of selecting a **subset** of the most relevant features from the training dataset for the machine learning task at hand.

- **Choose the most relevant** or most informative features (with predictive power) to use and **remove irrelevant, redundant**, or **noisy** ones.
  - Desire only features that have a significant effect on the model's output
  - Goal: maximize relevance and minimize redundancy
- Helps to reduce the number of features to a manageable subset
  - Can improve learning performance, accuracy, interpretability, and cost

Feature Selection Methods



Python packages `mlxtend` and `scikit-learn` have methods for these, and others.

Filter Method

Select features based on **1 or more performance measure(s)**.

- Use a hypothesis test to evaluate
- Independent of which ML algorithm is used

Examples:

- Select features with high correlation to the target variable (Pearson's Correlation Coefficient)
- Remove features with high correlation to each other (Pearson's Correlation Coefficient)
- Run a chi-square test of independence to see if there is a dependence between a categorical feature variable and the categorical target variable

Filter Method Python Example

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, chi2, f_classif

df = pd.read_csv('heart.csv')

# Show what all the feature options are
print('All features:', df.columns.tolist())

# Pull out the target variable to be evaluated separately
X = df.drop('output', axis=1)
y = df['output']

# Select the top k features, using a chi-squared test (for classification)
selector = SelectKBest(chi2, k=2)
X_new = selector.fit_transform(X, y)

print('Selected features:', X.columns[selector.get_support()].tolist())
```

```
All features: ['age', 'sex', 'cp', 'trtbps', 'chol', 'fbs', 'restecg', 'thalachh', 'exng', 'oldpeak', 'slp', 'caa', 'thall', 'output']
Selected features: ['thalachh', 'oldpeak']
```

Wrapper Method

Select the features that **perform the best** for the ML algorithm being used.

- Search through feature subsets using a search strategy (and possibly a stopping criteria).
- Evaluate each subset based on how the ML algorithm performs with it.
  - Need to choose a relevant metric/classifier

General Process:

1. Create possible feature subsets
2. Train on each subset
3. Choose the subset that performs best

Exhaustive Feature Selection (EFS)

Exhaustive Feature Selection is one possible wrapper strategy.

- Train a different ML model for **all possible feature subsets**, then evaluate.
- Brute force approach
- For a small number of features, it's fine
  - Ex. Dataset with 4 features
    - ~2^4 = 16 possible subsets
    - Train 16 different ML models with the different subsets
    - See which one did the best
- Searching through all possible feature subsets is **computationally expensive**!
  - A training dataset with 100 features has ~10^{30} possible subsets
  - EFS is rarely used in practice for this reason

Wrapper Method: Non-Exhaustive Methods

2 alternative, and more common, wrapper methods:

- **Forward Selection**
  - Start with a **single feature** and **add** more features until you find the best performance
- **Backwards Elimination**
  - Start with **all features** and **remove** features until you find the best performance

Others to be aware of:

- Bi-directional Elimination (hybrid of above FS and BE)
- Recursive Elimination (basically BE but different performance criteria)
- Boruta algorithm (Tree-based method - *future topics*)

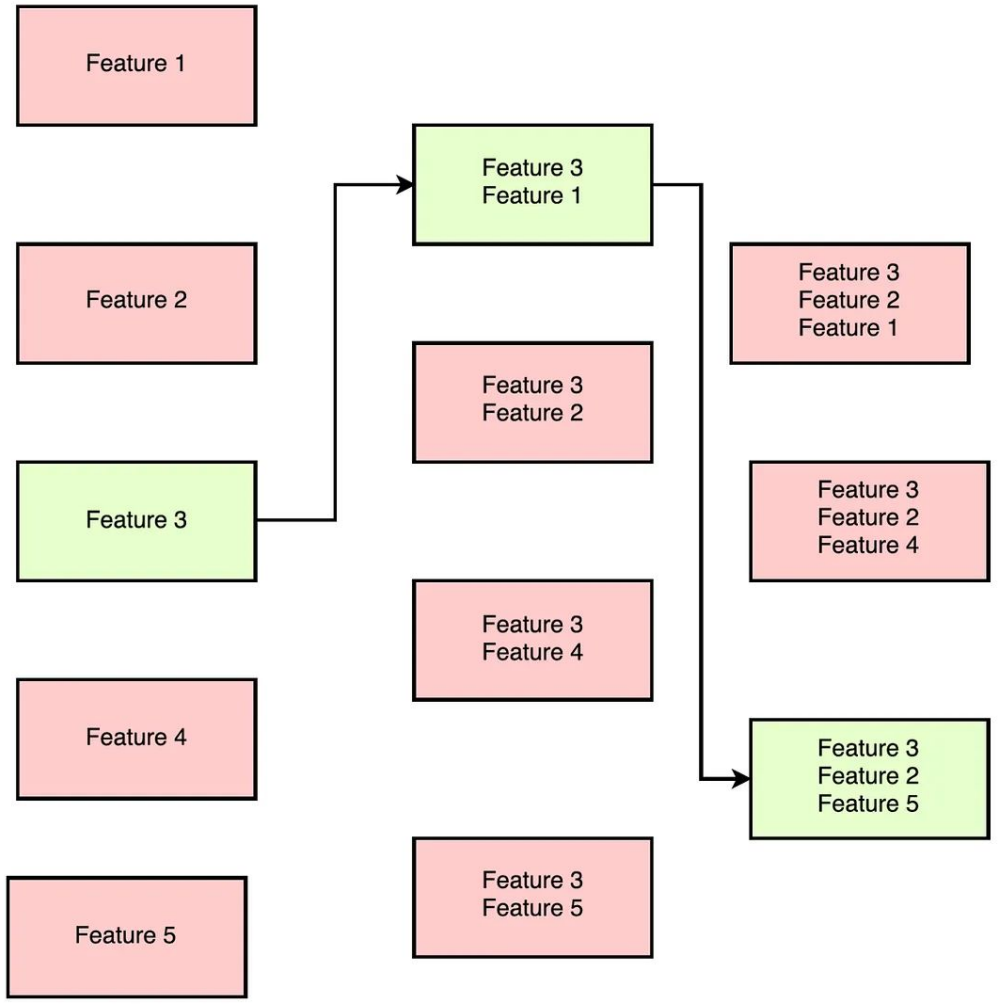
These are **Sequential Feature Selection** (SFS) methods, which is a greedy algorithm that iteratively adds or removes features to improve performance.

Forward Selection

Start with a **single feature**, then **add another feature** and repeat until you find the best performance

- Algorithm:
  - Train a different model on each feature in the dataset
  - Choose the feature that performed the best
  - If chosen stopping criteria is met, stop.
    - ex. Once model performance doesn't improve above a desired level, or after x features have been selected
  - Repeat, using the remaining features
- Advantages
  - Computationally more efficient than backward elimination (up next)
- Disadvantages
  - Doesn't account for between feature interactions
  - Stopping criteria choice can affect final subset of features

Forward Selection Example



Forward Selection Python Example

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier

df = pd.read_csv('Iris.csv')

X = df.drop('Species', axis=1)
y = df['Species']

# Create a KNN classification model
knn = KNeighborsClassifier(n_neighbors=3)

# Perform Forward Selection
sfs = SequentialFeatureSelector(knn, direction='forward')
sfs.fit(X, y)
selected = X.columns[sfs.get_support()].tolist()

print('Selected features:', selected)
print('Dimensions reduced: ', len(df.columns) - len(selected))
```

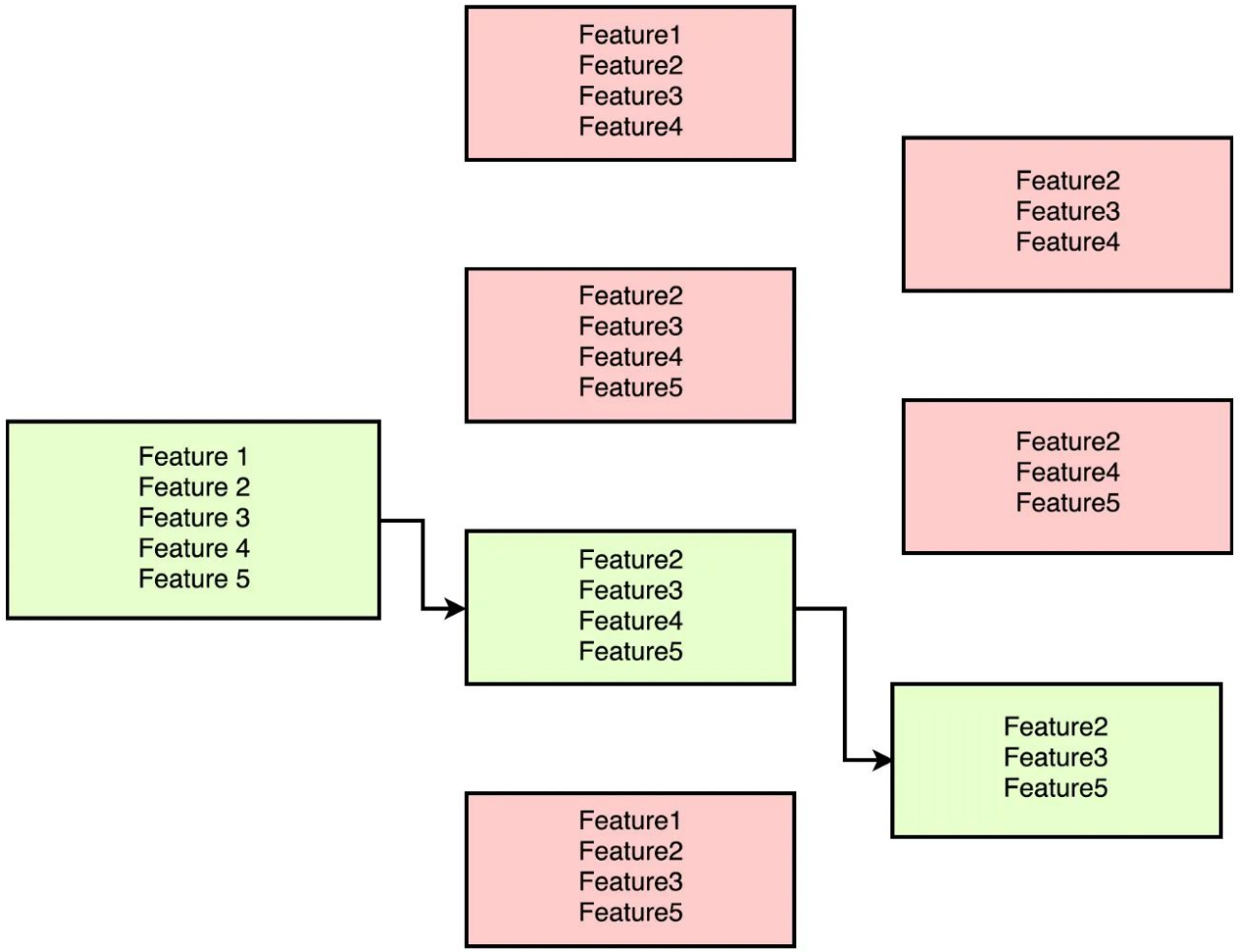
```
Selected features: ['SepalLengthCm', 'PetalWidthCm']
Dimensions reduced:  4
```

Backwards Elimination

Start with **all features**, then **remove features** until you find the best performance

- Algorithm:
  - Train a model on all features in the dataset
  - Generate all possible feature subsets by excluding each of the remaining features (one at a time)
  - Remove the feature that performed the worst
  - If chosen stopping criteria is met, stop.
    - ex. Once model performance doesn't degrade below desired level, or after x features have been selected
  - Repeat, using the remaining features
- Advantages
  - Accounts for between feature interactions
- Disadvantages
  - Computationally less efficient than forward selection
  - Stopping criteria choice can affect final subset of features

Backwards Elimination Example



Backwards Elimination Python Example

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier

df = pd.read_csv('Iris.csv')

X = df.drop('Species', axis=1)
y = df['Species']

# Create a KNN classification model
knn = KNeighborsClassifier(n_neighbors=3)

# Perform Forward Selection
sfs = SequentialFeatureSelector(knn, direction='backward')
sfs.fit(X, y)
selected = X.columns[sfs.get_support()].tolist()

print('Selected features:', selected)
print('Dimensions reduced: ', len(df.columns) - len(selected))
```

```
Selected features: ['SepalLengthCm', 'PetalLengthCm', 'PetalWidthCm']
Dimensions reduced:  3
```

Embedded Method

“Embed” feature selection into ML learning algorithm.

- Combines advantages of the Filter and Wrapper methods
- Main types (*future topics*): Regularization, Tree, Lasso (L1), Ridge (L2)
- Algorithm:
  - Train a model
  - Derive feature importance
  - Select the features that are ranked best and add, or ranked worst and remove
- Advantages
  - Much more computationally efficient than Wrapper and EFS
- Disadvantages
  - Not all algorithms can embed feature selection

This is a **Dynamic Feature Selection** method.

Summary

Method	Technique	Examples
Filter	Select features based on 1 or more performance measure(s).	Correlation, Chi-square test, ANOVA
Wrapper	Add or remove features iteratively or recursively to improve performance	Forward Selection, Backward Elimination
Embedded	Embed feature selection into the ML learning algorithm	LASSO, Ridge, Tree-based

Comparison of Feature Selection Methods

Method	Pros	Cons
Filter	Fast and efficient. Algorithm independent. Easy to interpret/understand why a feature is chosen.	Limited interaction of features with the model. Requires that you pick the right performance metric.
Wrapper	Optimizes for the model. Flexible - many possible approaches and evaluation techniques. Good balance between statistical measures of Filter and relying on a specific algorithm as with Embedded.	Requires effective evaluation criteria. More prone to overfitting. Computationally inefficient, depending on technique used.
Embedded	Computationally more efficient than wrapper methods. Optimizes for the algorithm.	Can be harder to interpret and understand why features were chosen. Not all ML algorithms can be embedded.

Feature Extraction

Feature Extraction

Feature extraction is the process of **extracting (deriving) features** from the original dataset, preserving the original relationships and most significant information, and mapping the data onto a new feature space in order to **reduce dimensionality**.

Feature Extraction

Goal: compress the data but maintain the most relevant information

Method: Use a mapping function from n-dimensional space to m-dimensional space (where m < n)

Mapping Function Approaches

- linear transformation
- piecewise linear functions
- nonlinear functions
- neural networks

Example

- Principal Component Analysis (PCA): *future topic*
  - Very popular unsupervised data compression algorithm

Feature Selection vs. Feature Extraction

Feature Selection	Feature Extraction
Chooses most relevant features	Creates new features from existing ones.
Original features are maintained	Features are transformed onto a new feature space